

Doctor Appointment Booking System

Statement about the Problem:

The healthcare sector is rapidly evolving, with an increasing demand for a streamlined and efficient appointment scheduling experience. This project aims to address the challenges faced by both patients and healthcare providers in managing and accessing appointment-related information. The primary objective is to design and develop a comprehensive **Doctor Appointment Booking System** that includes separate modules/interfaces for patients, doctors, and administrators.

The system will enable patients to **book, reschedule, or cancel appointments**, while doctors can manage their schedules and availability. Additionally, the platform will provide real-time notifications, secure patient-doctor communication, and integration with electronic health records. The development of this system will ensure a **user-friendly, secure, and efficient** healthcare booking process. The successful implementation of this project will improve **patient access to healthcare**, optimize doctor schedules, and enhance overall healthcare service delivery.

Why this particular topic is chosen?

The choice of creating a **Doctor Appointment Booking System** is driven by several compelling reasons and considerations:

- **Improving Healthcare Access:** Traditional appointment booking methods are often time-consuming and inefficient. This system enhances accessibility by allowing patients to book appointments online easily.
- **Enhancing Patient Experience:** Convenience and timely access to healthcare services are crucial. This application streamlines appointment scheduling, reminders, and doctor-patient communication, leading to improved patient satisfaction.
- **Efficient Doctor & Schedule Management:** Doctors can manage their availability, appointments, and patient history more efficiently, reducing scheduling conflicts and optimizing their workflow.
- **Data Management & Security:** Healthcare generates vast amounts of sensitive data daily. This system will securely store and manage patient records, ensuring compliance with privacy regulations and easy data access for authorized users.
- **Remote & Emergency Access:** Patients can book or cancel appointments remotely, ensuring flexibility. Additionally, emergency slots can be managed for urgent medical needs.

By implementing this system, healthcare services become more streamlined, patient-centered, and accessible to a wider population.

Objective of the Doctor Appointment Booking System:

The main objective of this project is to develop a feature-rich online appointment booking system for healthcare services. The platform will offer a range of functionalities to streamline appointment scheduling, patient management, and doctor availability tracking.

1. **Appointment Scheduling & Management:** To provide an easy-to-use system for patients to book, reschedule, or cancel appointments with healthcare professionals.
2. **Patient Information Management:** To record and manage essential patient details, ensuring accurate and secure data storage.
3. **Doctor Availability & Scheduling:** To allow doctors to set their availability, manage appointments, and avoid scheduling conflicts
4. **Specialization & Department Categorization:** To organize doctors based on their specializations, making it easier for patients to find the right doctor
5. **Admin Panel:** To manage appointments, patient records, doctors, and system configurations efficiently.

Scope of the Doctor Appointment Booking System Project:

The scope of this project includes the following key features:

1. **User Registration & Profile Management:** Patients and doctors can create and manage their profiles securely.
2. **Doctor Listings & Specialization Details:** Patients can browse through doctors based on specialty and availability.
3. **Appointment Booking System:** Patients can schedule, reschedule, or cancel appointments.
4. **Filter Options:** Users can search doctors based on specialization, fees.
5. **Secure Payment Options:** Patients can pay for consultations securely via multiple payment methods.
6. **Admin Panel for Management:** Admins can manage users, doctors, schedules, and system settings.

Methodology:

The "Waterfall methodology" will be used to complete this doctor appointment booking system. The Waterfall methodology is a straightforward approach that involves five phases:

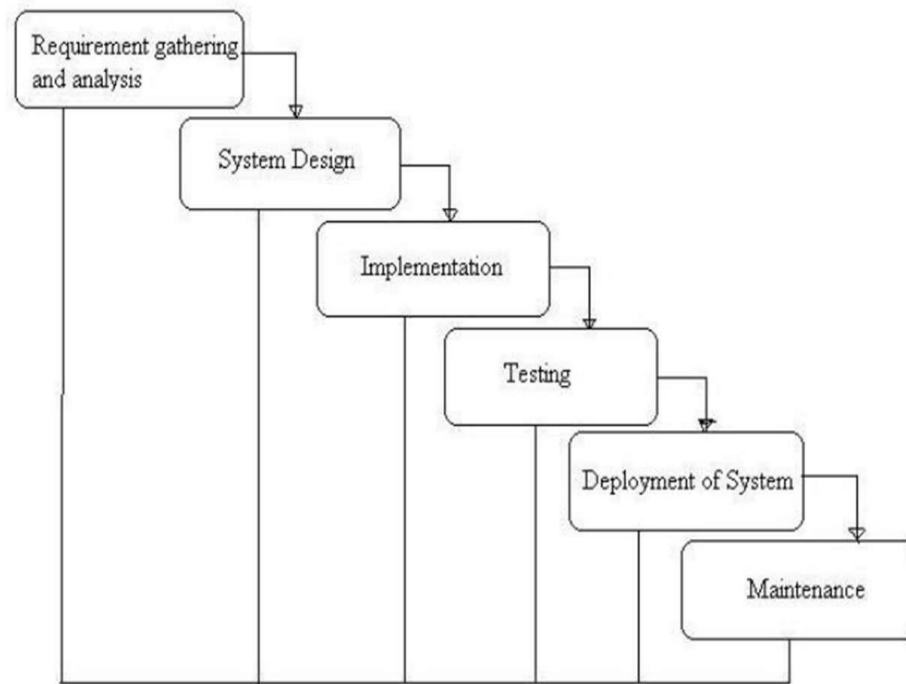
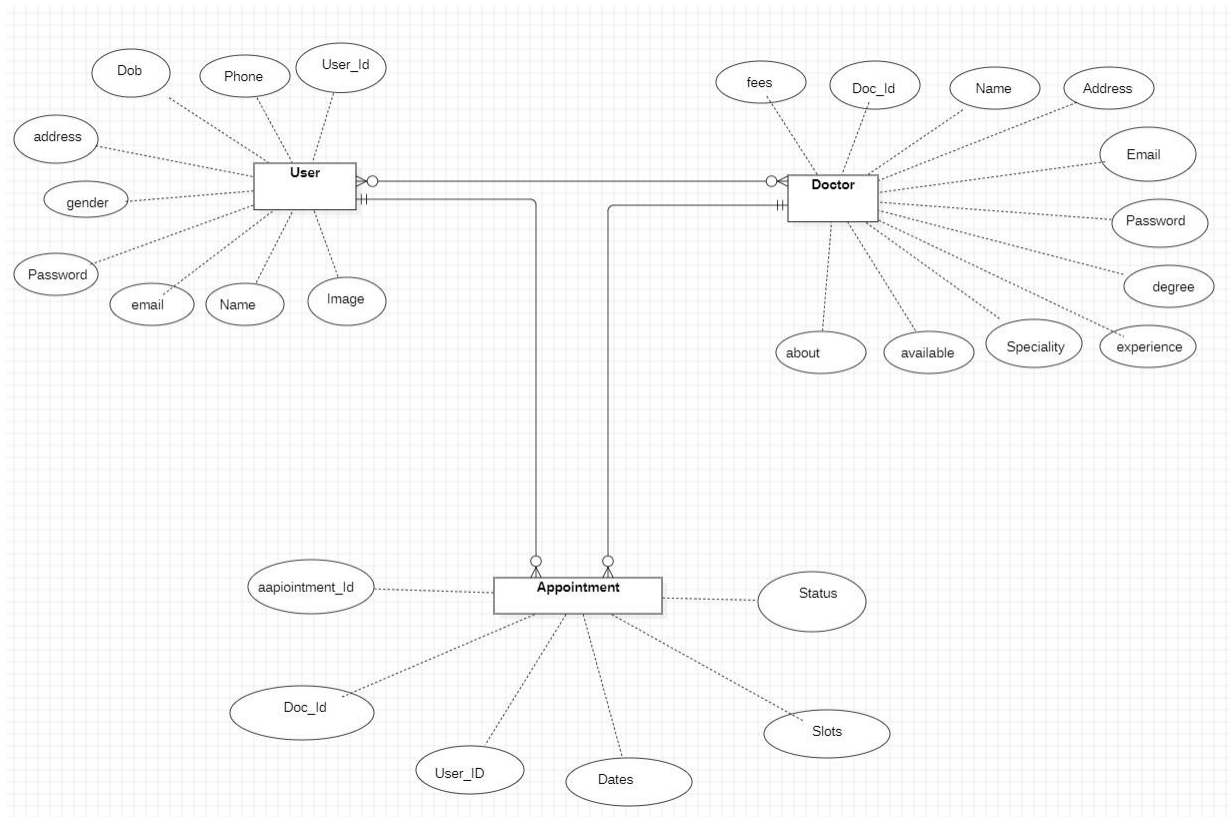
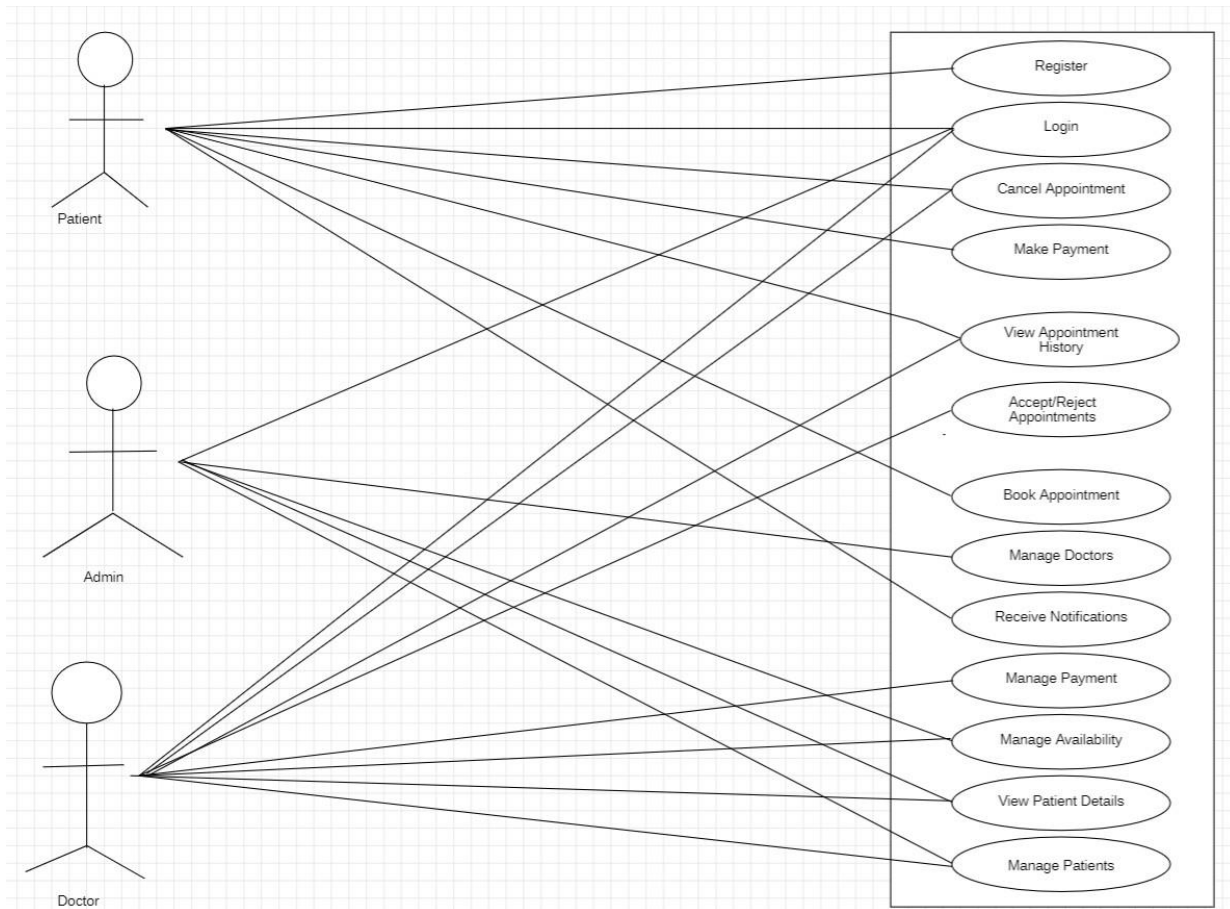


Fig.1 Waterfall Model

ER DIAGRAM:-

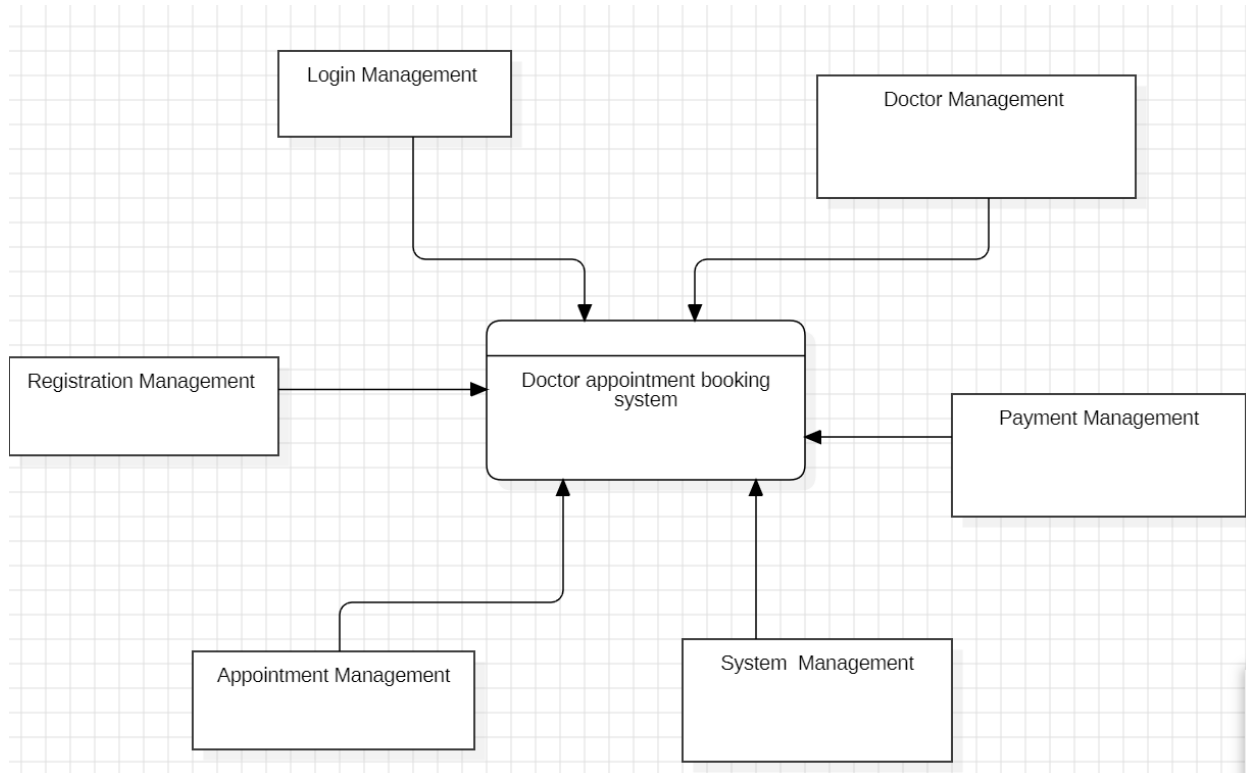


Use case :

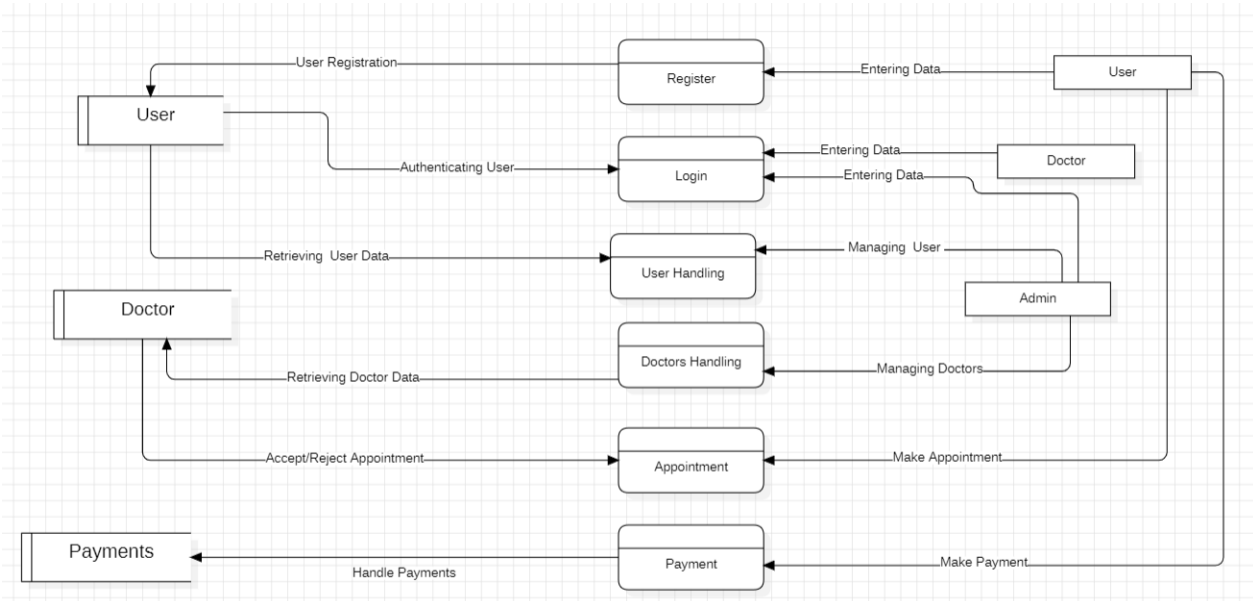


DFD

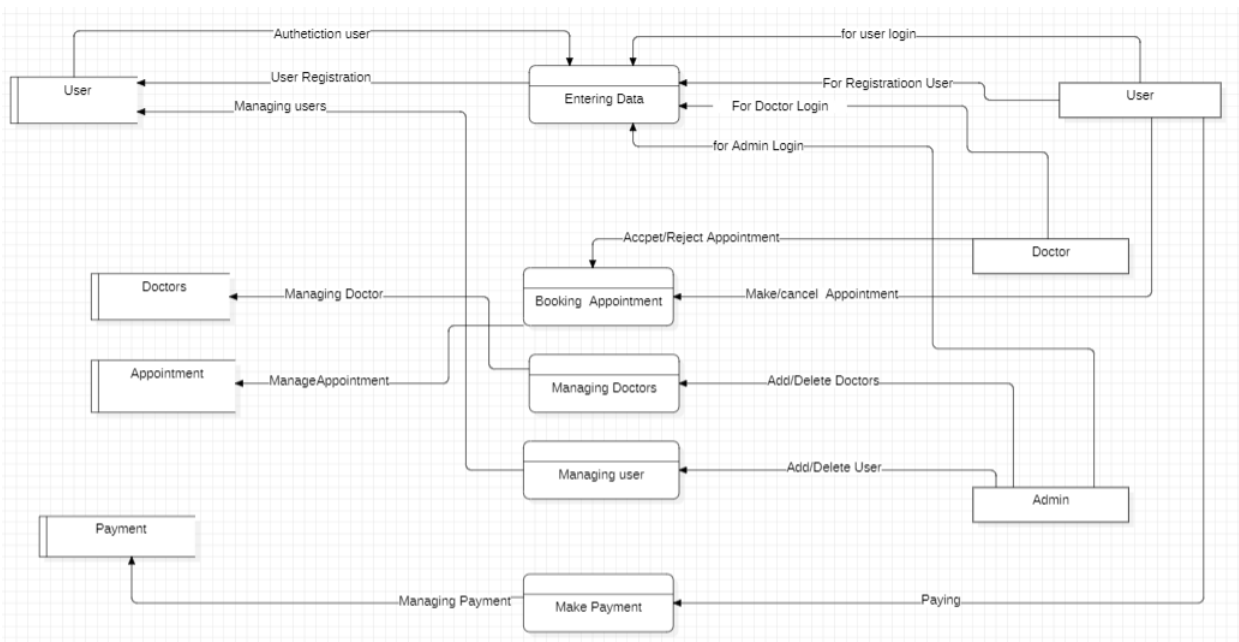
0-LEVEL-DFD:



1-LEVEL-DFd:



2-LEVEL-DFD:



Data Dictionary

Table 1: user

S.no	Field name	Field type	Constraint
1	_id	String	Primary key
2	name	String	Not Null
3	email	String	Not Null, Unique
4	password	String	Not Null
5	phone	String	Unique
6	address	String	Default: {line1: "", line2: ""}
7	gender	String	Default: "Not Selected"
8	dob	String	Default: "Not Selected"

Table2: Doctor

S.no	Field name	Field type	Constraint
1	_id	ObjectId	Primary Key
2	name	String	Not Null
3	email	String	Not Null, Unique
4	password	String	Not Null
5	image	String	Not Null
6	speciality	String	Not Null
7	degree	String	Not Null
8	experience	String	Not Null
9	about	String	Not Null
10	available	Boolean	Not Null
11	fees	Number	Not Null
12	address	Object	Not Null
13	date	Number	Not Null
14	slots_booked	Object	Default: { }

Table 3: Appointment

S.no	Field name	Field type	Constraint
1	userId	String	Primary key
2	docId	String	Not Null
3	slotDate	String	Not Null
4	slotTime	Object	Not Null
5	docData	Object	Not Null
6	userData	Object	Not Null
7	amount	Number	Not Null
8	date	Number	Not Null
9	cancelled	Boolean	Default: false
10	payment	Boolean	Default: false
11	isCompleted	Boolean	Default: false

Hardware & Software to

be used: Hardware:

- Processor: Intel Core i3
- RAM: 4GB
- Hard disk: 1 TB (minimum 80GB)

Software:

- Operating System: Windows 7, 10
- Front-end: HTML, CSS, JavaScript, React
- Server-side: Node.js, Express.js
- Database: MongoDB
- IDE: Visual Studio Code
- Project type: Web application

Testing Technologies used:

The project involves two main types of testing:

- **Unit Testing:** Focuses on verifying the correctness of small, isolated pieces of code within the backend and frontend. It helps catch and rectify errors in the early stages of development.
- **Manual Testing:** Essential for evaluating user interfaces and overall user experiences. It assesses aspects that may be challenging to automate, such as visual design, usability, and subjective user interfaces.
- **UAT(User Acceptance Testing):** Execute User Acceptance Testing (UAT) to confirm that the final product meets all specified requirements and is ready for deployment.

Contribution of the Doctor Appointment Booking System Project:

A doctor appointment booking system can bring significant benefits to the healthcare industry. It streamlines the appointment scheduling process, reducing paperwork and minimizing administrative burdens on medical staff. Patients can easily browse available slots, book appointments leading to an improved overall experience. The system enables remote access for patients, ensuring convenience and continuity of care. Additionally, developing this system allows for the implementation of robust security measures to protect patient data and ensure compliance with healthcare regulations.

MAIN REPORT

CHAPTER 1

OBJECTIVES & SCOPE OF THE PROJECT

1.1 Objective of the Project

The objective of this Major Project is to design and develop an enhanced **Doctor Appointment Booking System**, named "**Priscripto**", which specializes in facilitating seamless and efficient scheduling of medical consultations. With the increasing reliance on digital platforms for healthcare access and the specific challenges associated with appointment management — such as real-time availability, and secure data handling — *Priscripto* aims to deliver a highly functional, user-centric online solution tailored to meet these requirements.

Priscripto is designed not just as an appointment portal, but as a comprehensive healthcare access platform that integrates advanced features to significantly improve patient experience, streamline administrative tasks for clinics and hospitals, and provide a secure environment for managing medical data and online interactions. This project focuses on developing the following key functionalities to address the needs of both end-user and administrator providers:

Admin Panel for Efficient Platform Management

A crucial component of any healthcare appointment system is the ability to efficiently manage operations from the backend. *priscripto* provides administrators with a powerful, user-friendly Admin Panel designed to oversee and control all core functionalities of the platform. This centralized dashboard allows admins to:

- **Dashboard Overview**
 - Provides a high-level snapshot of current platform activity including total doctors, appointments, and registered patients.
 - Uses visual tools like graphs and charts to track trends and performance over time.
 - Alerts notify admins of pending actions or critical system issues.
- **Appointments Management**
 - Enables viewing, filtering, and managing all appointments across the platform.
 - Admins can confirm, reschedule, or cancel appointments as needed.

- Displays complete appointment details: doctor, patient, date, time, and reason for visit.
- Ensures smooth scheduling and reduces the likelihood of no-shows.
- **Doctor Registration and Profile Management**
 - Allows adding new doctors with comprehensive profile details (name, specialization, experience, consultation fee, availability, contact info).
 - Supports editing, deactivating, or removing doctor profiles when necessary.
 - Maintains an accurate and up-to-date directory of available doctors.
- **Doctors List View**
 - Displays all active and inactive doctors in a structured, searchable table format.
 - Allows quick access to view, update, or manage doctor profiles.
 - Includes key data such as appointment count, patient ratings, and feedback.
 - Helps evaluate doctor performance and patient satisfaction levels.

Secure Payment Processing with Stripe Integration

Security is paramount in online healthcare transactions. *Priscripto* integrates Stripe as its payment gateway, ensuring:

- **Encryption and Tokenization**
 - All payment data is encrypted and tokenized, preventing exposure of sensitive information during processing.
 - Ensures end-to-end security for each transaction.
- **Multiple Payment Options**
 - Supports a variety of payment methods including credit/debit cards, UPI, and digital wallets (e.g., Google Pay, Apple Pay).
 - Provides flexibility and convenience for users based on their preferences.
- **Fraud Detection and Security Compliance**
 - Utilizes Stripe's intelligent fraud detection tools to monitor and block suspicious activity.

- Fully compliant with **PCI DSS** standards, protecting both patient data and platform integrity.

Optimized User Experience (UX)

An intuitive interface and seamless navigation are essential to user retention and satisfaction. priscipto focuses on delivering an optimized user experience through:

- **Advanced Search and Filtering**
 - Users can easily search for doctors based on name, specialization, availability, location, and consultation fee.
 - Filtering helps patients find the most suitable healthcare provider quickly and efficiently.
- **Responsive Design**
 - The platform is fully optimized for desktops, tablets, and mobile devices.
 - Ensures a consistent, user-friendly experience across all screen sizes and browsers.
- **Personalized Recommendations**
 - Uses user activity and preferences to suggest relevant doctors, specialties, or health services.
 - Increases engagement and helps patients discover new or suitable options more effectively.

Scope of the Doctor Appointment Booking System Project

The **Doctor Appointment Booking System** is designed to streamline the process of scheduling medical consultations by providing an efficient and user-friendly digital platform for patients, doctors, and administrators. The primary goal of this project is to facilitate easy access to healthcare services through an online system that enables patients to find suitable doctors, book appointments, make secure payments, and manage their interactions with healthcare providers—all from a single interface. The system will also assist doctors in managing their schedules and patient interactions, while providing administrative tools for effective management and control of the platform.

The detailed scope of the project includes the following key components:

1. User Registration and Profile Management

One of the foundational features of the system is secure user registration and account management for both patients and doctors. Each user must be able to create an account with appropriate role-based access.

- **Patient Registration:** Patients can sign up by providing basic personal details such as name, contact information, gender, and date of birth. A verification process (via email or OTP) ensures authenticity.
- **Doctor Registration:** Doctors can register by entering their professional qualifications, specialties, certifications, clinic address, and availability schedule. Their accounts may require verification by an administrator before becoming active.
- **Login & Authentication:** Users can securely log in using email/username and password. Additional security features such as two-factor authentication may be included.
- **Profile Management:** Both patients and doctors can update their profile details, including contact information, profile pictures, and password resets.
- **Medical History:** Patients can optionally maintain a history of their medical conditions, allergies, past consultations, and treatments for better continuity of care.

2. Doctor Listings and Specialization Details

The system will provide an organized and searchable listing of all registered doctors, categorized by their area of expertise, location, and availability.

- **Doctor Profiles:** Each doctor's profile will include detailed information such as full name, qualifications, years of experience, areas of specialization, languages spoken, consultation fees, clinic address, and reviews from patients.
- **Availability Display:** Patients can view real-time available time slots for each doctor to make informed booking decisions.

3. Appointment Booking System

The core functionality of the system revolves around seamless appointment management. Patients can conveniently schedule consultations with doctors at a time that suits them.

- **Appointment Scheduling:** Patients can book appointments by selecting a preferred doctor, available date, and time slot.
- **Calendar Integration:** The system displays available slots and prevents double-booking through synchronized calendar views.
- **Appointment Modifications:** Patients can reschedule or cancel appointments with proper validation and policy-based restrictions (e.g., cancellation must be done 24 hours in advance).
- **Appointment History:** A record of past and upcoming appointments is maintained for both patients and doctors for future reference.

5. Admin Panel for Management and Oversight

An administrative backend is critical for ensuring the smooth operation of the platform. The admin panel provides tools to monitor, control, and manage users and system activities.

- **User Management:** Admins can view, edit, approve, or deactivate patient and doctor accounts. They can also assign roles and manage user permissions.
- **Doctor Verification:** Admins are responsible for reviewing and approving doctor profiles to ensure that only qualified professionals are listed.
- **Appointment Monitoring:** The admin can oversee all bookings, check for overbookings, and view appointment statistics.

Payment Gateway Integration – Stripe

To provide a seamless and secure payment experience, the Doctor Appointment Booking System will be integrated with **Stripe**, a globally trusted and robust online payment processing platform. This integration ensures that all financial transactions are handled efficiently and with the highest standards of security, transparency, and user convenience.

1. Secure Transactions

Stripe uses advanced encryption technologies and is fully **PCI DSS (Payment Card Industry Data Security Standard) compliant**, ensuring that all payment data transmitted through the platform

is protected against unauthorized access. This guarantees a high level of security for both the users and the system, reducing the risk of data breaches and financial theft.

2. Multiple Payment Options

Stripe supports a wide range of payment methods, allowing users to choose the most convenient option for them. The system will accommodate:

- **Credit Cards** (e.g., Visa, MasterCard, American Express)
- **Debit Cards**
- **Digital Wallets** (e.g., Apple Pay, Google Pay)
- **Bank Transfers** (in regions where supported)

This flexibility ensures greater accessibility and convenience for users with diverse payment preferences.

3. Fraud Detection and Prevention

The integration leverages **Stripe Radar**, a sophisticated fraud detection engine powered by machine learning. This feature:

- Automatically flags suspicious or high-risk transactions,
- Helps reduce chargebacks and payment fraud,
- Offers real-time analysis and mitigation tools to administrators.

By protecting the platform from fraudulent activities, it helps maintain user trust and safeguards business operations.

4. Transaction Tracking and Transparency

Stripe provides real-time transaction monitoring tools and a detailed dashboard for both users and administrators. Key benefits include:

- **Instant payment confirmation** and email notifications,
- **Automated invoice generation** after each successful transaction,

- **Historical payment logs** for users to view and download,
- Easy tracking of refunds or disputed transactions.

This transparency fosters user confidence and simplifies financial management for both doctors and patients.

5. Seamless Integration and Scalability

Stripe's API-first approach allows smooth integration with the booking platform's backend and frontend interfaces. It also supports:

- Recurring billing (for subscription-based services),
- Localization (multi-currency support and localized checkout),
- Scalability to handle large volumes of transactions as the platform grows.

CHAPTER 2

Theoretical Background Definition of problem

2.1 Theoretical Background

The healthcare industry, like many others, is undergoing rapid digital transformation, with online doctor appointment booking systems emerging as a crucial component of modern healthcare delivery. These systems leverage principles from the e-commerce domain—such as convenience, accessibility, and personalization—to improve patient experience and streamline clinical operations.

The Shift to Digital Healthcare Services

The widespread adoption of digital channels for healthcare access has been driven by several key factors:

- **Accessibility and Availability:** Unlike traditional appointment scheduling, which often requires in-person visits or lengthy phone calls during office hours, digital booking systems allow patients to view doctor availability and schedule appointments at any time. This enhances access to care, especially in remote or underserved areas.
- **User-Centric Convenience:** Online booking eliminates the need for physical presence, reduces waiting times, and provides patients the autonomy to choose suitable timeslots, specialties, and preferred physicians. This mirrors the customer-centric design of e-commerce platforms and aligns with rising expectations for convenience in all aspects of life.
- **Data-Driven Personalization:** By leveraging patient data such as medical history, past appointments, and preferences, modern systems can recommend appropriate specialists or follow-up appointments. This not only enhances the patient journey but also improves clinical outcomes through continuity of care.

Security and Privacy in Doctor Appointment Booking system

With the increasing digitization of healthcare services, **security and privacy** have become top priorities for both patients and healthcare providers. Online doctor appointment booking systems manage highly sensitive information, including personal health details, appointment histories, and payment data. As such, they must uphold stringent security standards to maintain trust, ensure patient safety, and comply with legal and regulatory requirements. Key elements of a secure digital healthcare system include:

- **Secure Payment Gateways:** For systems offering paid consultations or online billing, secure payment integration is essential. Platforms often use trusted services such as **Stripe**, known for robust encryption, PCI DSS compliance, and fraud detection capabilities. Features like **tokenization**, which replaces sensitive card information with unique identifiers, help protect financial data and reduce the risk of breaches.
- **Data Protection and Encryption:** Healthcare platforms must implement strong **data encryption** protocols for both storage and transmission of patient data. Role-based access controls, regular security audits, and secure authentication methods (e.g., two-factor authentication) are critical for safeguarding sensitive medical information. Compliance with legal frameworks such as **HIPAA** (in the U.S.) and **GDPR** (in Europe) ensures that patient data is handled with care and in accordance with privacy laws.

User-Centric Design for Optimal Experience

A successful digital doctor appointment booking system must be not only secure but also **highly user-focused**. User experience (UX) has become a critical component in patient engagement, satisfaction, and long-term platform adoption. In the context of healthcare, a well-designed UX enhances trust, reduces friction, and ensures that patients can access care with ease.

Effective UX design for healthcare platforms includes:

- **Intuitive Navigation:** Patients should be able to easily browse doctor profiles, check availability, and book appointments without confusion or unnecessary steps. A clear and logical interface supports users of all age groups and digital skill levels.

- **Fast and Responsive Interfaces:** The platform should load quickly and work seamlessly across mobile devices, tablets, and desktops. Given the growing preference for mobile health solutions, responsive design is essential for meeting patients where they are.
- **Accessible and Inclusive Features:** A user-centric design also means being inclusive. Features such as readable fonts, language selection, contrast settings, and support for screen readers ensure accessibility for all users, including those with disabilities.
- **Efficient Search and Filtering:** Patients should be able to search for doctors based on specialty, location, available time slots, insurance coverage, and other relevant criteria. Easy filtering empowers users to make informed decisions quickly.
- **Feedback-Driven Improvements:** Incorporating user feedback into the design process allows the system to evolve in line with patient needs and expectations. Regular updates based on real user behavior can significantly enhance usability and satisfaction.

By focusing on these aspects of UX, healthcare platforms can improve appointment conversion rates, reduce missed appointments, and foster stronger patient-provider relationships through a seamless digital experience.

2.2 Definition of Problem

In the increasingly competitive landscape of digital healthcare, a basic doctor appointment booking system is often insufficient to meet the expectations of patients and the operational demands of healthcare providers. The original version of the appointment system lacked several essential features required to deliver a smooth, secure, and user-centric experience. These shortcomings directly impacted patient satisfaction, administrative efficiency, and the system's ability to compete with more advanced healthcare platforms. Below are the key problems identified, which this project aims to address:

Lack of Real-Time Appointment Management

- **Inefficient Scheduling and Updates:** Without real-time synchronization of appointment slots, patients frequently encountered outdated or double-booked schedules. This led to missed appointments, confusion, and a loss of trust in the system's reliability.

- **Delayed Availability Updates:** Healthcare providers could not instantly update their availability or block off time slots, resulting in discrepancies between actual and displayed availability. This hindered the system’s ability to reflect real-time changes and negatively affected patient experiences.
- **Limited Administrative Control:** The platform lacked a user-friendly interface for doctors and administrative staff to manage their schedules, patient records, and appointment logistics. This caused operational delays and increased the burden on healthcare personnel who needed to manually coordinate appointments.

Security Gaps in Payment Processing

- **No Secure Payment Gateway:** The initial version of the appointment booking system did not include a secure and integrated payment gateway for paid consultations. This posed a significant barrier to usability, especially as many patients now expect to book and pay for services online—whether for in-person visits or telemedicine sessions. Without secure payment processing, the platform lacked credibility and professionalism.
- **Privacy and Safety Concerns:** The absence of essential security measures such as **encryption, fraud prevention tools, and compliance with healthcare and payment data regulations** created concerns among users about the safety of their personal and financial information. Patients were understandably reluctant to provide payment details without visible assurances of security. This not only weakened user trust but also exposed the platform to potential **data breaches** and **legal risks** related to mishandling of sensitive information.

Project Objective to Resolve Identified Problems

This upgraded project aims to address the limitations of the original doctor appointment booking system by building a **comprehensive, secure, and user-focused healthcare platform**. The new system is designed to enhance both the patient experience and administrative efficiency, ensuring timely, safe, and accessible healthcare services. The following objectives directly respond to the identified issues:

- **Real-Time Appointment Management:** The platform will feature a dynamic scheduling system that allows doctors to update their availability in real time and

prevents double-bookings. Patients will see accurate time slots and receive instant confirmations, reducing missed appointments and scheduling conflicts.

- **Secure Payment Integration with Stripe:** The platform will integrate **Stripe** to manage all payment transactions securely. This will address privacy and fraud concerns through data encryption, PCI DSS compliance, and tokenized payment processing, ensuring patients can confidently pay for services online.
- **Streamlined Back-End Appointment and Patient Management:** Administrative users and healthcare providers will benefit from improved tools to manage schedules, patient records, and appointment statuses. This will lead to greater efficiency, reduced workload, and better service delivery.
- **User-Centric Patient Experience:** The system will incorporate intuitive navigation, personalized provider suggestions based on patient history, and responsive design for all devices. These features will ensure a smooth, accessible, and engaging experience for all users.

CHAPTER 3

System Analysis & Design vis-a-vis User Requirements

3.1 System Analysis

In developing the upgraded doctor appointment booking system, a comprehensive analysis was conducted to identify the **core functionalities, user expectations (frontend), and administrative needs (backend)**. This analysis focused on improving accessibility, scheduling efficiency, data security, and the overall digital healthcare experience. The following outlines the system requirements and strategies for addressing them:

Frontend User Requirements:

- **Ease of Use:** Patients require a clean, responsive, and intuitive interface that makes it easy to search for doctors, view profiles, and book appointments. Smooth navigation, minimal loading times, and mobile compatibility are essential to ensure a positive user experience across all devices.
- **Profile Viewing:** Users expect detailed profiles for each doctor, including specializations, qualifications, availability, clinic location (or online consultation options), consultation fees, and patient reviews. Filtering options based on specialization, availability, and location will help users find suitable doctors efficiently.
- **Appointment Booking and Availability Viewing:** A user-friendly appointment booking module is necessary, where patients can view real-time availability and select preferred time slots. Integration with calendar systems and appointment reminders enhances scheduling efficiency and reduces missed appointments.
- **Secure Payment and Booking Confirmation:** For platforms offering paid consultations, a secure and simple checkout process is critical. This includes options for patients to securely enter payment details, review appointment summaries, and receive confirmation via email or SMS. The use of a secure gateway like **Stripe** will ensure transaction safety and build user trust.

Backend Administrative Requirements

- **Efficient Schedule and Provider Management:** Administrators and healthcare providers require real-time control over appointment slots, availability updates, and consultation durations. This ensures that patients always see accurate availability and reduces scheduling conflicts.
- **Appointment Tracking and Management:** Admins need tools to manage and monitor all booked appointments—from creation to completion or cancellation. This includes

oversight of no-shows, rescheduling, and patient follow-up, ensuring an organized workflow for providers and clinics.

- **User and Provider Account Management:** The system must allow administrators to manage both patient and doctor accounts. This includes reviewing registration details, monitoring system usage, managing feedback or complaints, and providing support when needed.
- **Analytics and Reporting Tools:** Backend users should have access to data dashboards that provide insights into appointment trends, patient satisfaction, consultation durations, and platform usage. These metrics are crucial for improving service delivery and operational decisions.

Security Requirements

- **Secure Payment via Stripe:** For paid consultations, Stripe will be used to process payments securely. Stripe ensures encryption, PCI DSS compliance, and tokenized transactions, minimizing the risk of data breaches and increasing patient confidence.
- **Secure Data Handling:** Sensitive data—including personal health information, appointment history, and contact details—must be stored and transmitted using robust encryption protocols. Access controls, data anonymization where necessary, and compliance with healthcare data protection regulations (e.g., HIPAA or GDPR) are essential for safeguarding patient trust and legal compliance.

3.3 System Design

Frontend Design:

- **Technology Stack:** The user interface is developed using **React**, offering a fast, responsive, and dynamic experience across devices. **Tailwind CSS** is employed for a clean, accessible, and modern design that adapts fluidly to mobile, tablet, and desktop screens.
- **Key Components:**
 - **Doctor Directory:** A searchable and filterable listing of doctors, where users can view detailed profiles including specializations, availability, consultation types (in-person or online), and fees.
 - **Appointment Booking Module:** Enables patients to view real-time availability and book appointments directly from a doctor's profile. Patients receive immediate confirmation and appointment details via email or SMS.
 - **Secure Payment Page:** A dedicated checkout interface for paid consultations, offering a simple, streamlined payment process integrated with **Stripe**. It includes

options to review appointment details, enter payment information securely, and receive confirmation.

Backend Design:

- **Technology Stack:** Built using **Node.js** and **Express.js** to manage server-side logic efficiently. This setup supports handling user authentication, appointment scheduling, secure data transactions, and system notifications.
- **Database Integration:** **MongoDB** is used as the primary database due to its flexibility and scalability. It effectively stores and manages complex data such as user profiles (patients and doctors), appointment schedules, availability slots, and transaction records.
- **Admin Panel:** A dedicated admin interface enables authorized users (e.g., clinic staff or system administrators) to manage doctor availability, oversee patient appointments, and access key analytics. This panel is critical for streamlining operations, providing real-time visibility into system activity, and maintaining service quality.

Database Design:

- **Database Technology:** **MongoDB** is selected for its flexibility and scalability, supporting collections for entities such as **Users, Doctors, Appointments, and Payments**.
- **Entity Relationships:**
 - **User Collection:** Stores patient information including name, contact details, login credentials (encrypted), appointment history, and saved preferences (e.g., preferred doctor or consultation mode).
 - **Doctor Collection:** Contains doctor profiles, specializations, availability schedules, consultation fees, and credentials.
 - **Appointment Collection:** Tracks all scheduled appointments, including patient and doctor references, date/time, status (booked, completed, cancelled), and consultation type (in-person or virtual).
 - **Payment Collection:** Stores transaction records for paid consultations, linked to users and appointments. Includes payment status, method, Stripe transaction ID, and timestamp.

Design Principles:

- **Modularity:** The system is built with modular components—such as appointment booking, doctor management, user authentication, and payment processing—that function independently but work seamlessly together. This allows for easier debugging, maintenance, and future enhancements.
- **Scalability:** The platform is designed to grow alongside user demand. New features like specialist filters, or multiple payment gateways can be added without disrupting core functionalities.
- **User-Centric Interface:** The frontend emphasizes ease of use with intuitive navigation, streamlined booking flows, and minimal steps from searching for a doctor to confirming an appointment. This design reduces user friction and enhances overall satisfaction.

CHAPTER 4

Methodology Adopted, System Implementation & Details of Hardware & Software Used, System Maintenance & Evaluation

4.1 Methodology Adopted

To develop the **Doctor Appointment Booking System**, the **Waterfall Methodology** was chosen for its structured, sequential approach. This methodology allowed for a well-defined process, where each development stage was completed before the next began. Given the critical nature of features such as real-time scheduling, secure data handling, and user management, the Waterfall model ensured clarity, traceability, and reduced risk of scope creep.

The methodology supported comprehensive documentation, clear milestones, and effective resource planning, which were essential in delivering a secure and user-friendly medical appointment system. The following stages were followed:

• **Requirements Gathering:**

This initial phase focused on collecting detailed requirements from both patients (end users) and administrators (clinic staff or system managers).

- **For users:** Features such as doctor search, viewing availability, booking appointments, and secure payment options were defined.
- **For administrators:** Requirements included managing doctor schedules, monitoring appointments, and accessing system analytics.

• **System Design:**

In this phase, wireframes and mockups were created for all essential pages, including:

- Doctor directory and profile pages
- Appointment booking interface
- User dashboard for tracking bookings
- Admin panel for managing doctors, appointments, and user data

Design considerations also included database schema planning and system architecture outlining the interaction between frontend, backend, and database layers.

• **Implementation:**

The system was divided into frontend and backend components, each developed to fulfill distinct functionality goals tailored to a healthcare appointment environment.

Frontend:

Developed using **React** and **Tailwind CSS** to deliver a responsive, accessible, and user-friendly interface across devices. Key features implemented on the frontend include:

- User account management (registration, login, password handling)
- Doctor listing and profile views, with filtering by specialty and availability
- Appointment booking and scheduling, with real-time time slot selection
- Patient dashboard, allowing users to manage appointments

Backend:

Built with **Node.js** and **Express.js**, the backend handles all server-side logic, including:

- User authentication and role-based access (patients and admins)
- Doctor and appointment management, including CRUD operations for schedules
- Secure data handling using **MongoDB** to store user profiles, doctor data, and appointment records

Payment Integration:

Stripe was integrated for processing consultation fees securely. Using API keys and tokenization, the system ensures that users' financial data is encrypted and protected during transactions, in compliance with **PCI-DSS standards**.

Testing:

Multiple testing phases were carried out to ensure the reliability and usability of the system:

- **Unit Testing:**
Individual components—such as appointment scheduling, doctor profile display, login system, and admin controls—were tested independently to verify correct functionality.
- **Integration Testing:**
End-to-end processes, including booking flow, data updates between the frontend and backend, and Stripe payment processing, were tested to ensure seamless interaction between modules.
- **User Acceptance Testing (UAT):**
Final testing was performed by a group of end users (both patients and administrators) to

validate that the application fulfilled real-world requirements in terms of performance, ease of use, and feature completeness.

These testing stages were essential in identifying and resolving issues before deployment, ensuring a stable and user-friendly platform.

Deployment:

- After successful testing, the application was deployed to a **live production environment**. Continuous monitoring tools were set up to track performance, identify bugs, and ensure platform availability.
- **Post-deployment performance testing** was conducted to assess:
 - System responsiveness under normal and high-traffic conditions
 - Booking efficiency and backend reliability during simultaneous user interactions

Each development and deployment phase was executed sequentially, with detailed evaluations and feedback loops in place. This allowed the development team to promptly address any detected issues and ensured a cohesive, robust final product.

4.2 System Implementation

The implementation of the **Doctor Appointment Booking System** was organized into three core components to ensure a reliable, user-friendly experience for patients and effective operational tools for administrators.

• Frontend Development:

Tools Used:

React was selected for its ability to build dynamic and responsive user interfaces, while **Tailwind CSS** provided a modern, mobile-friendly design.

Key Features Implemented:

- **Doctor Directory and Profiles:**
Allows users to browse and filter doctors by specialty, availability, consultation fees, and ratings.
- **Appointment Booking Interface:**
An intuitive UI where users can view available time slots and confirm appointments with a few clicks.
- **User Account Management:**
Enables users to register, log in, and manage their personal information and booking history.
- **Patient Dashboard:**
Offers a personalized interface where users can view upcoming appointments, cancel or reschedule them, and access past visit summaries.

• Backend Development:

Tools Used:

Built with **Node.js** and **Express.js** for handling server-side logic and routing. **MongoDB** was used as the NoSQL database for flexible and scalable data storage.

Core Functionalities:

- **User Authentication and Role Management:**
Ensures secure login and differentiated access rights for patients and administrators.
 - **Doctor and Appointment Management:**
Allows CRUD operations on doctor profiles and appointment slots, supporting real-time availability updates.
 - **Admin Panel:**
A secure interface for clinic staff to manage doctor schedules, review patient bookings, and access system analytics.
 - **Data Handling:**
MongoDB efficiently stores and retrieves records for users, doctors, and appointments, enabling seamless frontend-backend interaction.
-
- **Payment Gateway Integration:**
 - **Integration:**
Stripe was integrated for handling paid consultations. API keys and tokenization mechanisms were used to maintain secure transactions.
 - **Security:**
All financial data is encrypted and managed in compliance with **Payment Card Industry Data Security Standards (PCI-DSS)**, ensuring user trust and data protection.

4.3 Details of Hardware & Software Used

This section outlines the hardware and software specifications required for developing, testing, and deploying the **Doctor Appointment Booking System**.

• Hardware Requirements:

- **Processor:**
Intel Core i5 or equivalent processor was used for local development and testing, providing sufficient performance to handle real-time operations and parallel processes.
- **RAM:**
A minimum of **8GB RAM** was necessary to support efficient multitasking, including running the database server, backend APIs, and frontend development environment simultaneously.
- **Storage:**
At least **1TB of storage** capacity was used, with **100GB available** for storing project files, medical records, backups, appointment logs, and database data.

• Software Requirements:

Frontend:

- **React:**
Utilized for building responsive and interactive user interfaces for patients and administrators.
- **Tailwind CSS:**
Enabled consistent and mobile-friendly styling with utility-first CSS classes, accelerating the UI development process.

Backend:

- **Node.js & Express.js:**
Employed to develop scalable server-side logic and RESTful APIs, managing user sessions, appointments, and doctor availability.
- **MongoDB:**
Served as the primary NoSQL database, storing structured documents for users, doctors, appointment records, and schedules.

Payment Processing:

- **Stripe:**
Integrated for processing consultation payments securely. Tokenization and encryption techniques ensured that sensitive payment data remained protected and compliant with **PCI-DSS standards**.

4.4 System Maintenance & Evaluation

• System Maintenance:

After deployment, maintaining the **Doctor Appointment Booking System** is essential to ensure its security, efficiency, and continued reliability. The maintenance strategy includes:

- **Regular Updates:**
Frequent updates to dependencies, libraries, and the application's codebase are applied to patch vulnerabilities, enhance performance, and introduce new features.
- **Database Optimization:**
MongoDB indexes are reviewed and optimized periodically to maintain high performance during data retrieval, especially for appointment queries and availability lookups.
- **Server Monitoring:**
Monitoring tools such as **New Relic**, **Datadog**, or **PM2** are used to track system health, performance metrics, and uptime, allowing quick detection of issues.
- **Backup and Recovery:**
Automated backup routines are scheduled to protect patient, doctor, and appointment data. In the event of a system failure, rapid recovery procedures are in place to minimize downtime and data loss.

• System Evaluation:

To ensure continued improvement and alignment with user expectations, the system is regularly evaluated using the following strategies:

- **Performance Testing:**
The application is periodically stress-tested under various load scenarios (e.g., high concurrent appointment bookings) to ensure responsiveness and stability during peak hours.
- **User Feedback Collection:**
Feedback from both patients and administrative staff is collected through surveys and support channels. This information informs future updates—such as UI/UX enhancements or feature additions like telemedicine support.
- **Security Audits:**
Bi-annual security audits are conducted to identify and mitigate vulnerabilities, especially around sensitive data areas such as login credentials, personal information, and payment processing through **Stripe**.

Conclusion:

Through regular maintenance and ongoing evaluation, the **Doctor Appointment Booking System** is kept secure, efficient, and adaptable—ensuring that it continues to meet the evolving needs of healthcare providers and patients alike.

CHAPTER 5

Detailed Life Cycle of the Project

5.1 ERD, DFD

5.2 Input and Output screen Design

5.3 Process Involved

5.4 Methodology used testing

5.5 Test Report, Printout of the Report & Code Sheet

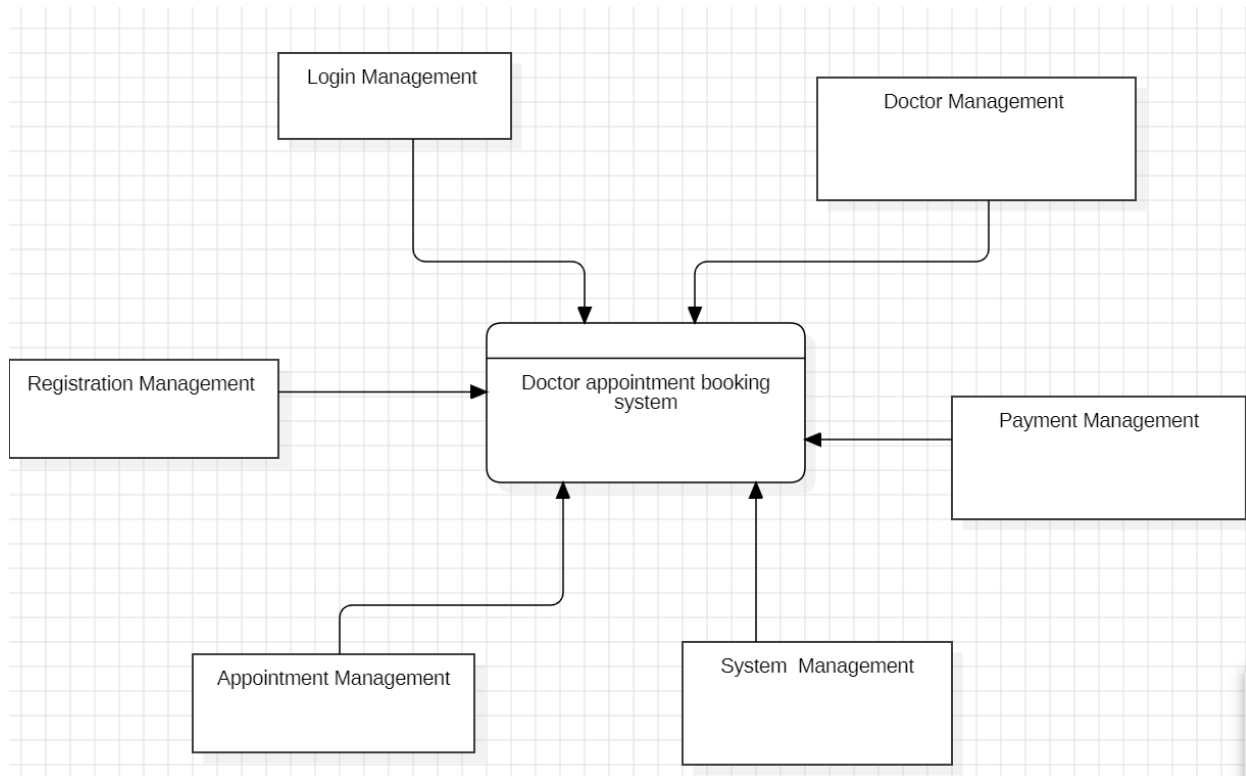
CHAPTER 5

Detailed Life Cycle of the Project

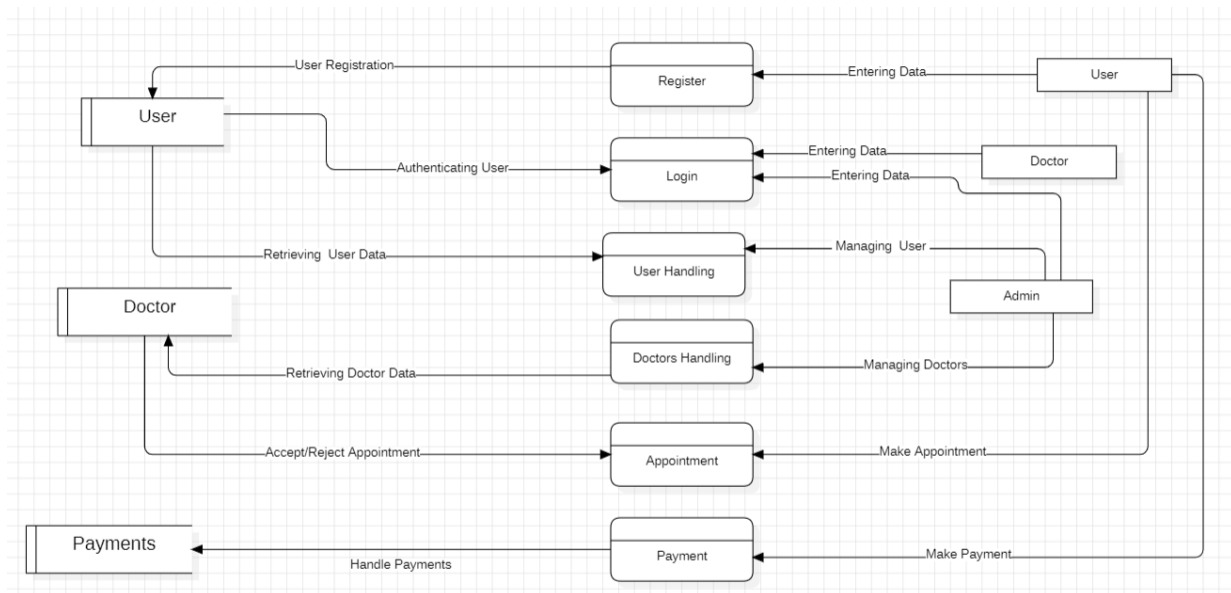
5.1 (DFD)

The Data Flow Diagram (DFD) will map the flow of information within the e-commerce system, illustrating how data moves from user interactions to backend processing.

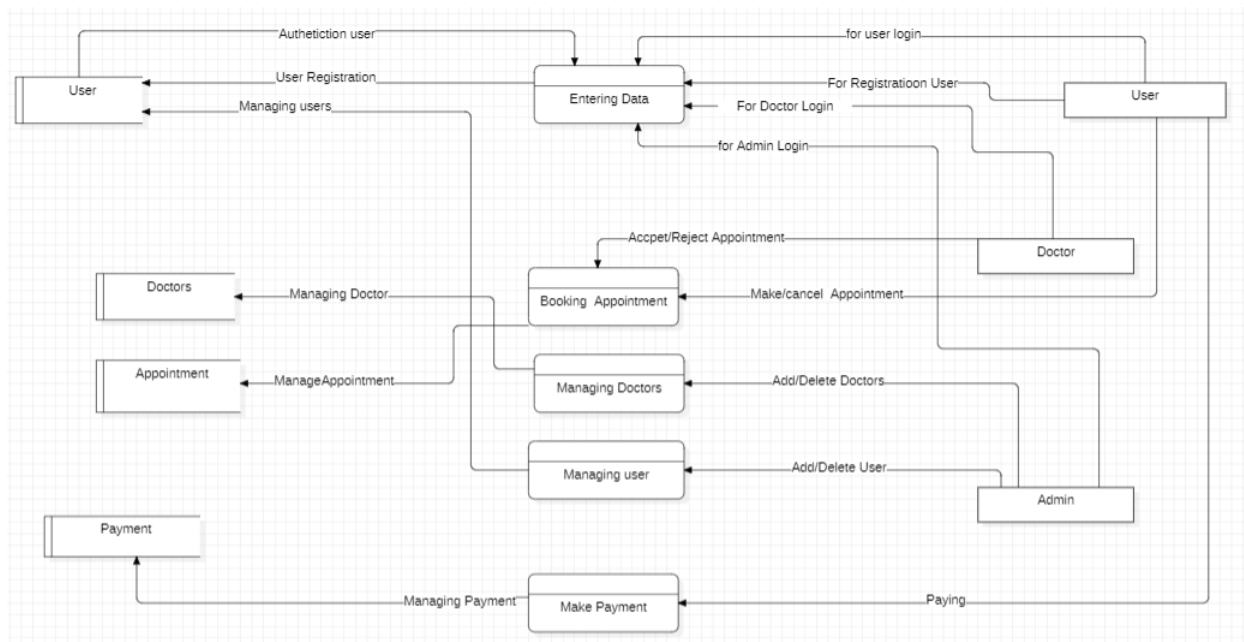
0-level DFD



1-level DFD



2-level DFD



Processes:

- **User Registration:**
Users input their personal information to create an account. This process collects and stores user data for future logins and profile management.
- **User Login:**
Registered users enter their login credentials to access the system. The system verifies the details to authenticate the user.
- **Doctor Login:**
Doctors use their login credentials to enter the system. Upon successful authentication, they gain access to manage appointments and respond to booking requests.
- **Admin Login:**
The administrator logs in to manage both doctors and users. This process authenticates admin access and grants system-level control.
- **Booking Appointment:**
Users select a doctor and request an appointment. This process sends the request to the doctor, who can then accept or reject the booking.

- **Manage Appointments:**
Doctors can view all appointment requests and either approve or cancel them. The appointment data is updated accordingly.
- **Managing Doctors:**
Admins can add new doctors, update their information, or remove them from the system. This ensures the doctor database remains current.
- **Managing Users:**
Admins can manage user accounts by adding new users, updating existing profiles, or deleting inactive users.
- **Make Payment:**
After booking an appointment, users proceed to the payment process. This securely handles the transaction and updates the payment record.

5.2 Input and Output Screen Design

Input and output screen design is crucial for creating a user-friendly interface that ensures efficient data entry, validation, and display. Proper design helps prevent errors, enhances usability, and improves the overall user experience

- **User Registration:**
The registration screen will contain fields for username, password, and email address. Data validation will check for:

Figure(Login Form)

Prescripto Home Doctors Contact About [Create Account](#)

Create Account

Please sign up to book an appointment

Full Name

Email

Enter a valid email address (e.g., user@example.com)

Password

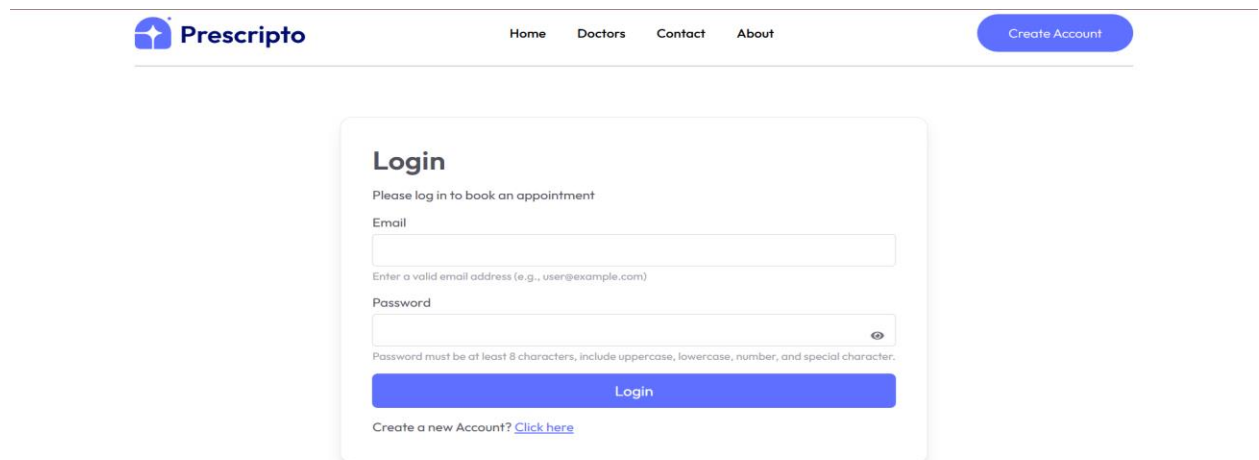
Password must be at least 8 characters, include uppercase, lowercase, number, and special character.

[Create Account](#)

Already have an account? [Login here](#)

- **Login Form:** This will include fields for • Email • Password • Password Recovery — to enhance user experience and support easy account access

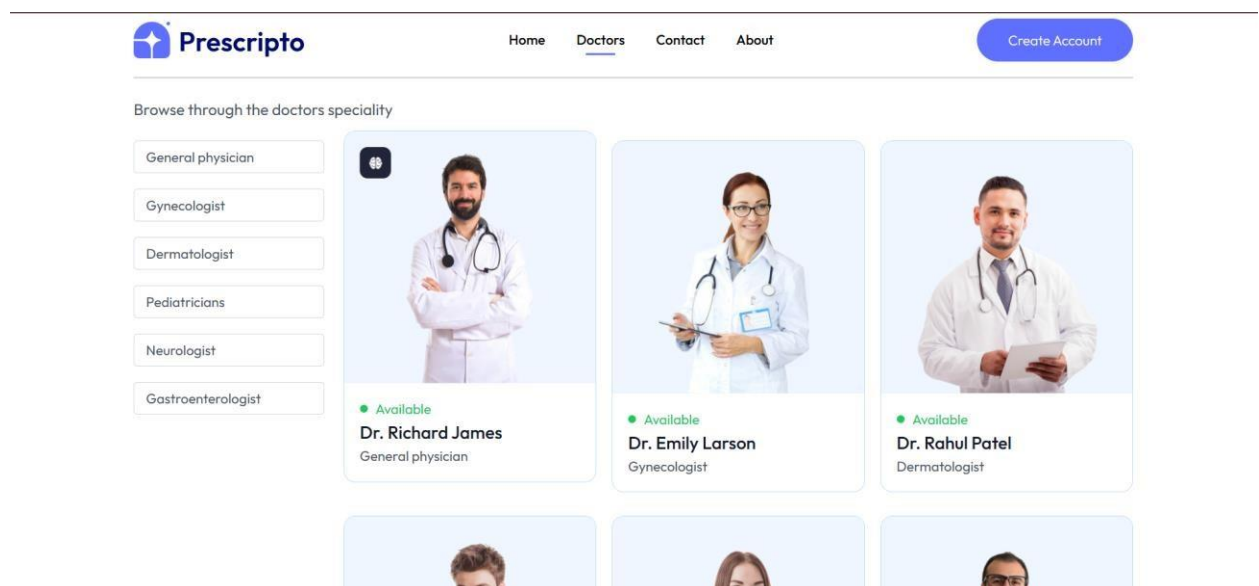
Figure(Login Form)



The image shows a web browser window with the Prescripto website. The navigation bar at the top includes the Prescripto logo, links for Home, Doctors, Contact, and About, and a blue 'Create Account' button. The main content area features a 'Login' form. The form has a title 'Login' and a subtitle 'Please log in to book an appointment'. It contains two input fields: 'Email' and 'Password'. Below the email field is a placeholder text: 'Enter a valid email address (e.g., user@example.com)'. Below the password field is a placeholder text: 'Password must be at least 8 characters, include uppercase, lowercase, number, and special character.' There is a blue 'Login' button and a link 'Create a new Account? [Click here](#)'.

- **Doctor Filter:** Users can filter doctors by • Specialty — using a dropdown menu to easily find the right medical expert.

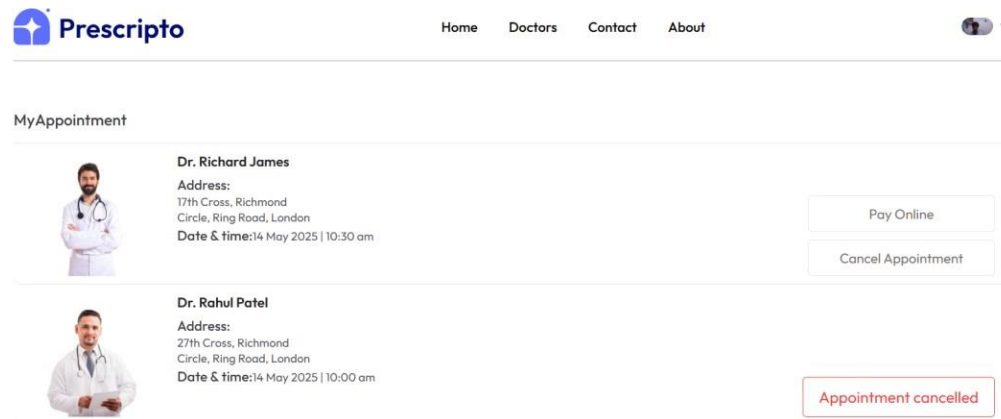
Figure(Doctor Filter)



The image shows a web browser window with the Prescripto website. The navigation bar at the top includes the Prescripto logo, links for Home, Doctors, Contact, and About, and a blue 'Create Account' button. The main content area features a 'Browse through the doctors speciality' section. On the left, there is a list of specialties: General physician, Gynecologist, Dermatologist, Pediatricians, Neurologist, and Gastroenterologist. On the right, there are three doctor profiles, each with a photo, a green dot indicating 'Available', and the doctor's name and specialty: Dr. Richard James (General physician), Dr. Emily Larson (Gynecologist), and Dr. Rahul Patel (Dermatologist). Below these profiles, there are three more placeholder images for doctors.

- **Appointment Management:** Users can • Book • Reschedule • Cancel appointments — ensuring flexibility and control over their bookings.

Figure(Appointment Management)



- **Contact Us:**

This section provides essential contact information and career opportunities for users and job seekers. It includes:

- **Office Address** – The physical location of the organization for correspondence or visits
- **Telephone Number** – A direct contact number for inquiries or support
- **Email Address** – An official email where users can send questions, requests, or feedback
- **Careers Button/Link** – A call-to-action for users interested in job opportunities, redirecting them to a careers page or job listings

Figure(Contact Us)

CONTACT US



OUR OFFICE

00000 Willms Station
Suite 000, Washington, USA

Tel: (000) 000-0000
Email: greatstackdev@gmail.com

CAREERS AT PRESCRIPTO

Learn more about our teams and job openings.

[Explore Jobs](#)

- **Manage My Profile:**

Users can update their personal information through a dedicated profile form. The editable fields include:

- **Name**
- **Profile Image**
- **Phone Number**
- **Date of Birth (DOB)**
- **Gender**
- **Address**

Real-time validation ensures data is entered in the correct format (e.g., valid phone number or proper date format).

Figure(My-Profile form)

 **Prescripto**

HomeDoctorsContactAbout





shitij parmar

CONTACT INFORMATION

Email Id:shitijparmar976@gmail.com

phone:8766325580

Address:A-1/144, sector-16
Rohini, delhi-110089

BASIC INFORMATION

Gender:Male

Birthday:2025-05-14T00:00:00.000Z

Edit

Doctor Listing (Admin Panel):

Admins can add, update, or remove doctor profiles from the system. This process involves entering detailed doctor information such as:

- Name
- Email
- Password
- Years of Experience
- Consultation Fees
- About Doctor (a short bio or profile description)
- Speciality

Once submitted, the doctor's data is stored in the database and made available to users for appointment booking and viewing. Real-time validation ensures that the entered data meets predefined formats and standards (e.g., valid email, numeric fees, minimum password strength).

Doctors can also be categorized by specialty to support search and filtering functions for patients.

Figure(Add-Doctor Form)


Prescripto
Dashboard Panel

Admin

Dashboard

Appointments

Add Doctor

Doctors List

Add Doctor



Upload doctor picture

Doctor name

Richard

Doctor Email

richadr@gmail.com

Doctor Password

Experience

3 Year

Fees

50

About Doctor

hi my name is richard

Speciality

Gastroenterologist

Education

MBBS

Address

address1

address2

Add Doctor

- **Output Design**

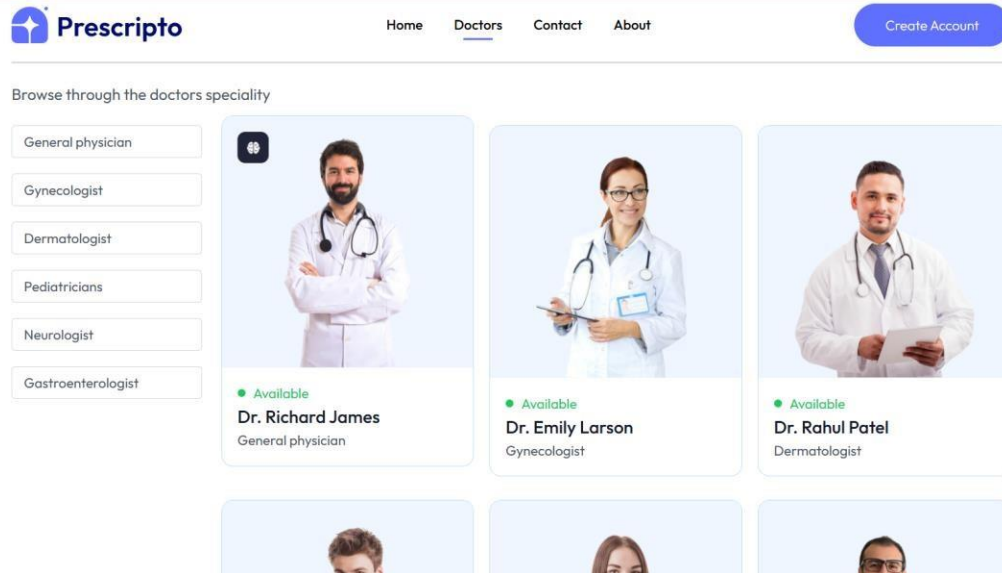
Output design focuses on how information is presented to users, ensuring that it is **visually appealing, well-organized, and easy to interpret**. The goal is to provide relevant data in a clear and efficient format that enhances user experience and supports informed decision-making.

Doctor Listings:

The system will display doctors in a **grid or card layout**, showcasing key details such as:

- **Availability**
- **Profile Image**
- **Name**
- **Speciality**

Figure(Doctor Listing)



- **Doctor Detail Page:**

The doctor detail page provides **in-depth information** about a specific doctor, helping users make informed decisions when booking an appointment. It includes:

- **Doctor's Name**
- **Profile Image**
- **Years of Experience**
- **Speciality**
- **Consultation Fees**
- **Description/Bio** – A short introduction about the doctor
- **Education/Qualifications**
- **Ratings and Reviews** (if available)

The layout is designed to present this information in a **clear and user-friendly manner**, often with a **“Book Appointment”** button prominently displayed. This page helps users evaluate doctors based on expertise, experience, and user feedback.

Figure(Doctor Detail page)



Dr. Richard James

MBBS - General physician 4 Year

About

Dr. Davis has a strong commitment to delivering comprehensive medical care, focusing on preventive medicine, early diagnosis, and effective treatment strategies. Dr. Davis has a strong commitment to delivering comprehensive medical care, focusing on preventive medicine, early diagnosis, and effective treatment strategies.

Appointment fee: \$50

• Doctor Management by Admin

The **Admin Panel** includes a dedicated section where administrators can fully manage all doctor-related data. This ensures that the system maintains an accurate and up-to-date list of available doctors for users to interact with.

Key Functions Admins Can Perform:

1. Add New Doctors

Admins can register new doctors into the system by filling out a form with the following details:

- Name
- Email
- Password (used for login access)
- Profile Image
- Speciality (e.g., Cardiologist, Dentist, Pediatrician)
- Years of Experience
- Consultation Fees
- Description/About the Doctor (short bio or professional summary)
- Education/Qualifications

2. Edit/Update Doctor Profiles

Admins can modify existing profiles if:

- A doctor changes their specialty or fees
- New qualifications or certifications need to be added
- There are errors in the original data
- A doctor wants to update their bio or contact info

3. Categorization & Search Optimization

- Admins can tag or categorize doctors by their specialties, helping users filter search results easily (e.g., "Show only Neurologists").

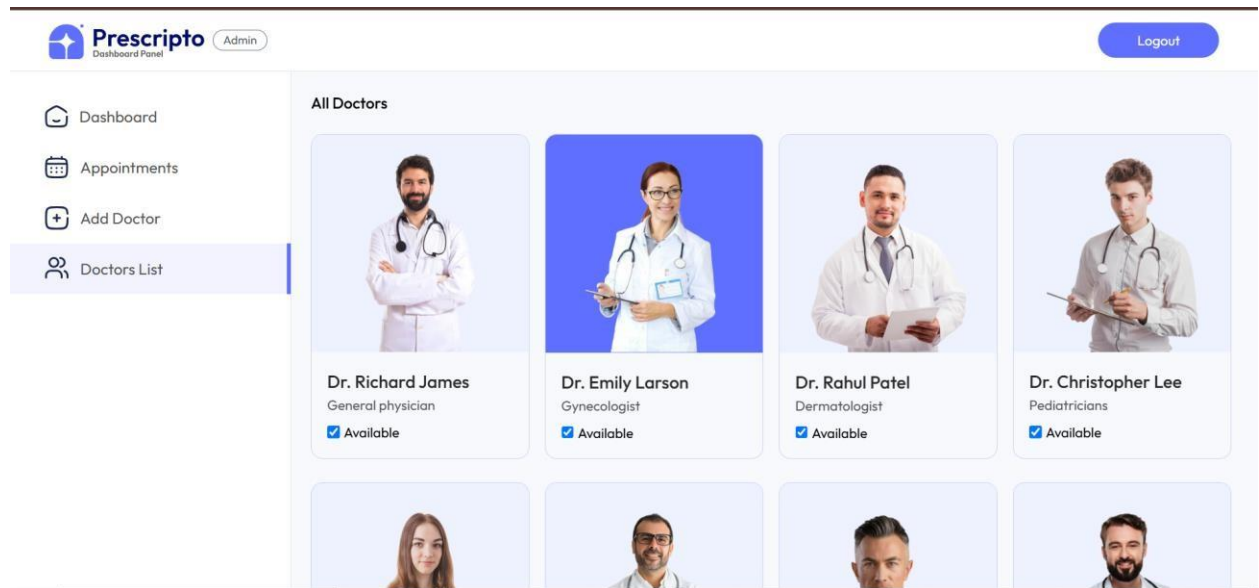
4. Monitor Doctor Activity

- Admins may have access to dashboards that show:
 - Number of appointments booked with each doctor
 - User ratings or feedback
 - Login frequency or activity logs

5. Set Availability

- Admins may also help manage time slots or availability calendars if the doctor doesn't handle it themselves.

Figure(Doctor Management)



Output designs will emphasize clarity and simplicity, ensuring that users can quickly access the information they need without unnecessary complexity.

5.3 Methodology Used – Testing

The testing methodology consists of several structured phases, each designed to assess different aspects of the platform to ensure it meets high standards of functionality, performance, and reliability.

Unit Testing

Unit testing focuses on testing individual components or units of code, such as login forms, doctor registration, and appointment booking.

- **Objective:** Identify bugs or defects in the smallest parts of the application before integrating them into larger modules.
- **Approach:** Each function or feature is tested independently in isolation using test cases to verify correct behavior.

Module Testing

After successful unit testing, module testing is performed to verify the functionality of grouped components such as user management, doctor listing, and payment processing.

- **Objective:** Ensure that each module functions correctly on its own and handles expected inputs and outputs effectively.
- **Approach:** Simulated scenarios and workflows are created to test each module's functionality comprehensively.

Integration Testing

This phase tests the interaction between different modules—such as frontend, backend, and database components—to ensure smooth and reliable communication.

- **Objective:** Detect and resolve issues that may arise when combining modules, ensuring seamless data exchange and functionality.
- **Focus Areas:**
 - Data retrieval and update between frontend forms and backend APIs
 - User login and session management
 - Appointment booking flow involving multiple modules

System Testing

In this phase, the entire system is tested as a whole to verify that it meets the specified requirements and works under real-world scenarios.

- **Objective:** Validate the overall functionality, performance, and reliability of the system before deployment.
- **Coverage:**
 - End-to-end workflows (e.g., user registration to appointment confirmation)
 - UI/UX elements
 - Functional behavior across different devices and browsers

White Box / Black Box Testing

White Box Testing

White box testing focuses on the **internal workings** of the application, including the structure, logic, and code implementation.

- **Objective:** To evaluate how internal operations of the software perform and to verify the correctness of each function and control flow.
- **Approach:** Developers/testers will write and execute test cases that check conditions, loops, paths, and data flows in the source code.

Black Box Testing

Black box testing involves evaluating the software **from the end-user's perspective**, without any knowledge of the underlying code or logic.

- **Objective:** To verify that the system meets functional requirements and handles input/output correctly.
- **Approach:** Testers will focus on the behavior of the application, checking if the inputs produce the expected outputs, without considering how those outputs are generated internally.

Acceptance Testing

User Acceptance Testing (UAT)

This phase ensures that the platform aligns with **business objectives** and fulfills **end-user needs** before final deployment.

- **Objective:** To confirm that the software is ready for release and meets all user expectations and business goals.
- **Participants:** Real users, stakeholders, or clients.
- **Methodology:**
 - UAT sessions will simulate real-life usage.
 - Users will perform standard tasks like booking appointments, editing profiles, and contacting support.
 - Feedback will be gathered to identify final tweaks or adjustments.
 - Any critical issues will be resolved before the system goes live.

5.5 Test Report, Printout of the Report & Code Sheet

A comprehensive set of test cases has been designed and executed to validate the core functionalities of the system. These test cases simulate real user scenarios, ensuring every feature operates as intended under expected and edge conditions.

Test Data

User 1:

- Name: John Doe
- Email: john.doe@example.com
- Password: Pass@123

Test cases

Table(Registration)

Serial No.	Test Case	Test Input	Expected Output	Pass/Fail
<u>1</u>	Valid registration	User 1 data	Successful registration, user redirected to Login Page	Pass
<u>2</u>	Missing required fields	Email missing	Error message: "Please fill out this field "	Pass
<u>3</u>	Invalid email format	Name: John, Email: johnexample.com	Error message: "Please include an '@' in the email address. 'john.doeexample.com' is missing an '@"	Pass
<u>4</u>	Weak password	Name: Jane, Password: 123	Error message: "Password must include uppercase, digits, symbols"	Fail
<u>5</u>	Duplicate email registration	Email: john.doe@example.com (already registered)	Error message: "User already exist with this email"	Pass
<u>6</u>	Missing required fields	Password Missing	Error message: "Please fill out this field "	Error message: "Please fill out this field "

Figure(Sample Data)

 **Prescripto**

[Home](#) [Doctors](#) [Contact](#) [About](#)

Create Account

Create Account

Please sign up to book an appointment

Full Name

Email

Enter a valid email address (e.g., user@example.com)

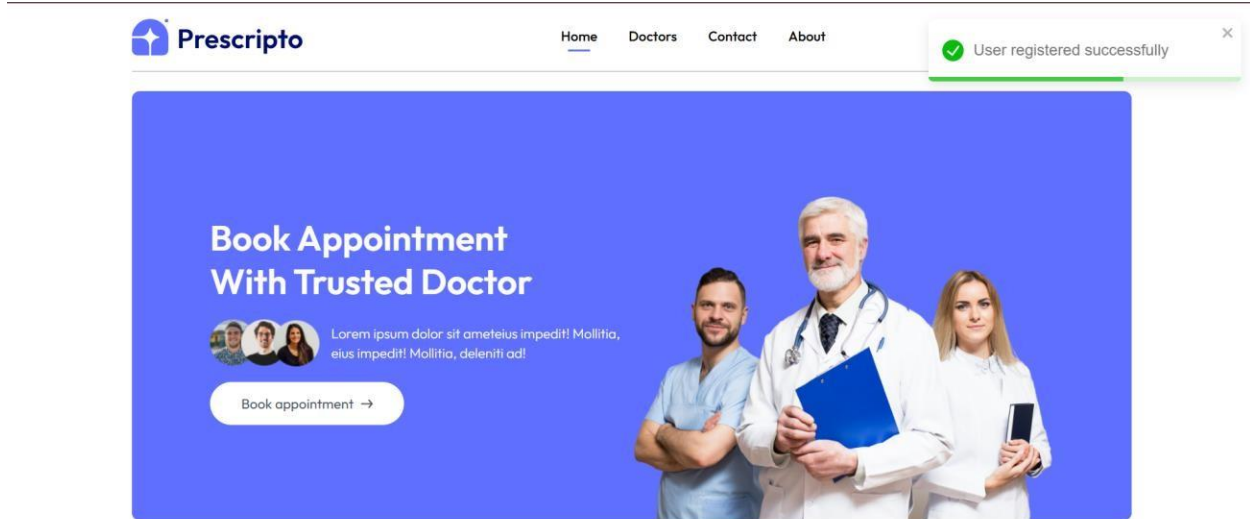
Password

Password must be at least 8 characters, include uppercase, lowercase, number, and special character.

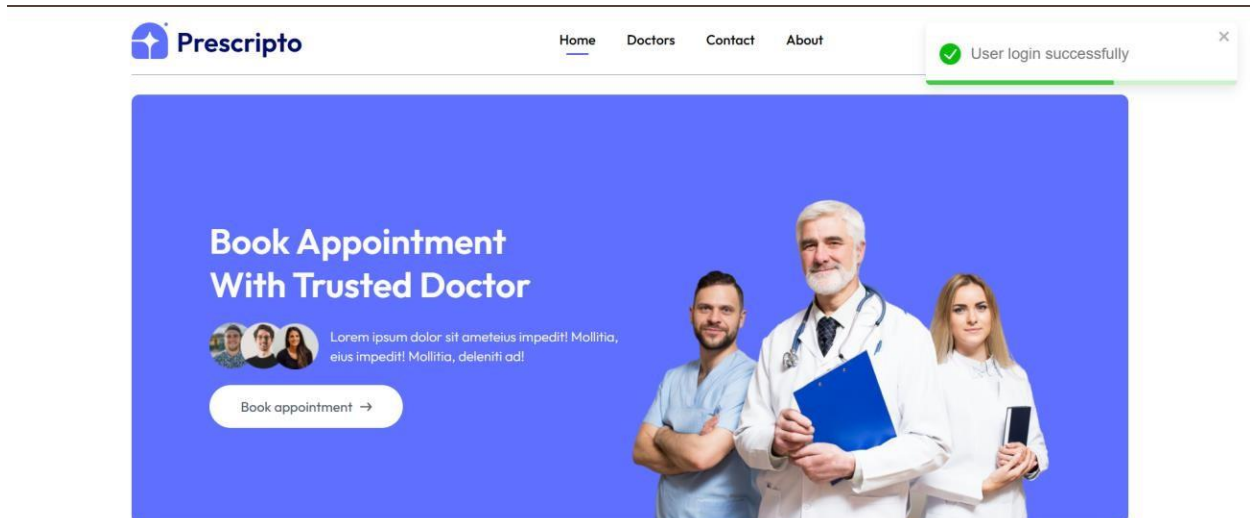
Create Account

Already have an account? [Login here](#)

Figure(User Registered)



Figure(Directed To Login Page)



Figure(Sample Data – Missing Email)

 Prescripto

[Home](#) [Doctors](#) [Contact](#) [About](#)

Create Account

Create Account

Please sign up to book an appointment

Full Name

Email

Enter a valid email address (e.g., user@example.com)

Password

Please fill out this field.

Password must be at least 8 characters, include uppercase, lowercase, number, and special character.

Create Account

Already have an account? [Login here](#)

Figure(Sample Data Missing - Password)

 Prescripto

[Home](#) [Doctors](#) [Contact](#) [About](#)

Create Account

Create Account

Please sign up to book an appointment

Full Name

Email

Enter a valid email address (e.g., user@example.com)

Password

Please fill out this field.

Password must be at least 8 characters, include uppercase, lowercase, number, and special character.

Create Account

Already have an account? [Login here](#)

Figure(Sample Data Wrong Email Format)

Create Account

Please sign up to book an appointment

Full Name

rishi

Email

rishi2003@gmail..com

Enter a valid email address

! is used at a wrong position in 'gmail..com'.

Password

Password must be at least 8 characters, include uppercase, lowercase, number, and special character.

Create Account

Already have an account? [Login here](#)

Figure(Directed To Home Page)

Book Appointment With Trusted Doctor



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Mollitia, eius impedit! Mollitia, deleniti ad!

Book appointment →

CHAPTER 6

Coding and Screenshots of the project

Coding

server Code

```
import express from 'express';

import cors from 'cors';

import 'dotenv/config';

import connectDB from './config/mongodb.js';

import connectCloudinary from './config/cloudinary.js';

import adminRouter from './routes/adminRoute.js';

import doctorRouter from './routes/doctorRoute.js';

import path from 'path';

import userRouter from './routes/userRoute.js';

// 🌐 App config

const app = express();

const port = process.env.PORT || 4000;

// 🔌 Connect to DB and Cloudinary

connectDB(); // MongoDB connected

connectCloudinary(); // Cloudinary connected


// 🛠 Middleware
```

```

app.use(express.json()); // Parse JSON

app.use(cors()); // Enable CORS

app.use('/uploads', express.static(path.join(process.cwd(), 'uploads'))); // Serve uploaded files

// 🚀 API endpoints

app.use('/api/admin', adminRouter); // Admin routes

app.use('/api/doctor', doctorRouter); // Doctor routes

app.use('/api/user', userRouter); // User routes

// 🏠 Root endpoint

app.get('/', (req, res) => {

  res.send('🌟 Server is working!');

});

// ⚠️ Error handling middleware app.use((err,
req, res, next) => { console.error(`❌ Error:
${err.message}`);

  res.status(500).json({ message: err.message });

});

// 🚀 Start server

app.listen(port, () => console.log(`✅ Server started on port ${port} 🚀`));

```

App Code

```
import React from "react";

import { BrowserRouter, Routes, Route } from "react-router-dom";

import { ToastContainer } from "react-toastify"; // Import ToastContainer

import "react-toastify/dist/ReactToastify.css"; // Import Toastify CSS

import Home from "./pages/Home";

import Doctors from "./pages/Doctors";

import About from "./pages/About";

import Contact from "./pages/Contact";

import MyProfile from "./pages/MyProfile";

import MyAppointments from "./pages/MyAppointments";

import Appointment from "./pages/Appointment";

import Navbar from "./components/Navbar";

import Login from "./pages/Login";

import Footer from "./components/Footer";

const App = () => {

  return (

    <BrowserRouter>

      <div className="mx-4 sm:mx-[10%]">

        <ToastContainer /> { /* Add ToastContainer here */ }

        <Navbar />

        <Routes>

          <Route path="/" element={<Home />} />


```

```

    <Route path="/login" element={<Login />} />

    <Route path="/doctors" element={<Doctors />} />

    <Route path="/doctors/:speciality" element={<Doctors />} />

    <Route path="/about" element={<About />} />

    <Route path="/contact" element={<Contact />} />

    <Route path="/my-profile" element={<MyProfile />} />

    <Route path="/my-appointments" element={<MyAppointments />} />

    <Route path="/appointment/:docId" element={<Appointment />} />

  </Routes>

  <Footer />

</div>

</BrowserRouter>

);

};

export default App;

```

Home Code

```

import React from 'react'

import Header from '../components/Header'

import Specialitymenu from '../components/Specialitymenu'

import TopDoctor from '../components/TopDoctor'

import Banner from '../components/Banner'

import Footer from '../components/Footer'

```

```
const Home = () => {  
  return (  
    <div>  
      <Header/>  
      <Specialitymenu/>  
      <TopDoctor/>  
      <Banner/>  
  
    </div>  
  )  
}  
  
export default Home
```

CHAPTER 7

Conclusion and Future Scope

7.1 Conclusion

The development of the doctor appointment booking system represents a significant advancement in the delivery of healthcare services, enabling clinics and hospitals to effectively meet the increasing demand for digital healthcare access. The platform is designed to streamline the appointment scheduling process through a user-friendly interface and a comprehensive set of features tailored to both patients and medical professionals.

By integrating modern technologies and a user-centered approach, the system ensures that patients can easily browse available doctors by speciality, view their profiles, and book appointments with convenience and confidence. Doctors and administrators benefit from a centralized backend that simplifies appointment management, user registration, and patient data handling.

Overall, this project enhances the efficiency and accessibility of healthcare services, improving the patient experience and optimizing clinic operations. It positions healthcare providers to adapt to digital transformation, ensuring better service delivery and greater patient satisfaction in today's connected world.

CHAPTER 8

References

1. GeeksforGeeks – <https://www.geeksforgeeks.org>
2. MongoDB – <https://www.mongodb.com>
3. Express.js – <https://expressjs.com>
4. React – <https://react.dev>
5. Node.js – <https://nodejs.org/en>
6. Tailwind CSS – <https://tailwindcss.com>
7. NPM – <https://www.npmjs.com>
8. Stack Overflow – <https://stackoverflow.com>
9. ChatGPT – <https://chatgpt.com>